

Control Flows, Scan

1. The Tagless Switch

A standard switch checks a specific variable (a "tag") against exact values. Go also offers a **Tagless Switch**.

- **How it works:** You write switch without giving it a variable at all. Instead, every case contains a full Boolean condition.

```
switch {  
case x > 1:  
    fmt.Printf("case1")  
case x < -1:  
    fmt.Printf("case2")  
default:  
    fmt.Printf("nocase")  
}
```

2. Loop Control: break and continue

Sometimes you need to manually interrupt a for loop based on a specific event. Go gives you two tools for this:

- **Break:** If a loop hits a **break statement**, the loop is completely destroyed. The program jumps out of the loop and moves on to the rest of the code.
 - **Example:** You are looping through a database to find a specific user. Once you find them, you **break the loop to stop wasting CPU cycles**.

```
for j := 0; j < 10; j++ {  
    if j == 5 { break }  
    fmt.Printf("hi ")  
}
```

- **Continue:** If a loop hits a **continue statement**, it stops executing the current iteration, but the loop itself stays alive. It simply jumps back to the top and starts the next round.
 - **Example:** You are processing a list of files, but one file is corrupted. You continue to skip the bad file and move on to the next one.

```
for j := 0; j < 10; j++ {  
    if j == 5 { break }  
    fmt.Printf("hi ")  
}
```

3. Taking User Input (fmt.Scan)

To make command-line tools interactive, you need to read what the user types on their keyboard. Go handles this using the `fmt.Scan` function.

- **How it works:** When your code hits `fmt.Scan`, the program literally pauses (blocks) and waits. It only resumes after the user types something and hits the **Enter** key.
- **The Pointer Requirement:** This is where your knowledge of Pointers comes in! To save the user's input, you must pass the memory address of your variable using the ampersand (&).
 - Example: `var appleNum int; fmt.Scan(&appleNum)` If you don't use `&`, the `Scan` function won't know where in the computer's memory to securely store the typed number.
- **Return Values:** `fmt.Scan` actually returns two pieces of background information: the number of items successfully scanned, and an error object (which will be `nil` if everything went perfectly).

Expert Takeaway:

Using **`fmt.Scan`**, combined with **infinite for loops** and **switch statements**, is exactly how you build robust, interactive **Command Line Interfaces (CLIs)**.

When writing utility scripts to automate your deployment pipelines or manage local server environments, you can drop into an **infinite loop**, prompt the user for an action, read it with **`Scan`**, process it cleanly with a **tagless switch**, and use **`break`** when they type **"exit"**. It is a classic, highly efficient architecture pattern!